Otimização de Pipeline MLOps para Manutenção Preditiva Industrial

Pedro Henrique Nunes Zanette

Atividade Prática — Engenharia de Dados e MLOps · Prof. MSc. Hugo Menezes Barra

Repositório: https://github.com/PedroZanette/atividade-manutencao-preditiva-mlops

1. Contexto e critério de decisão

Na TechIndustries, cada falha não detectada (Falso Negativo) causa uma parada não planejada de **R$ 15.000,00**, e cada alarme falso (Falso Positivo) gera uma manutenção preventiva desnecessária de **R$ 500,00**. O modelo a ser promovido é aquele com o menor custo total de erro nos três últimos meses de 2025 (2208 horas, 50 falhas):

$$\text{Custo Total (R\$)} = (FN \times 15.000) + (FP \times 500)$$

Como referência, foram calculados dois cenários sem modelo: **nunca alarmar** custa R$ 750.000,00 (50 falhas perdidas) e **alarmar todas as horas** custa R$ 1.079.000,00 (2158 alarmes falsos). Um modelo só gera valor para a empresa se custar menos que R$ 750.000,00.

2. Metodologia

Para isolar o efeito de cada mudança, os cinco modelos foram treinados em três cenários, todos com a mesma base, a mesma divisão temporal (treino jan–set, teste out–dez) e os mesmos hiperparâmetros do guia original:

Cenário	Features	Desbalanceamento	Objetivo
A	10 originais	class_weight / scale_pos_weight	Tarefa 1 (base)
B	17 (10 + 7 novas)	class_weight / scale_pos_weight	Tarefa 2 (Pergunta 1)
C	17 (10 + 7 novas)	SMOTE (sem pesos)	Tarefa 3 (pipeline otimizado)

Tarefa 2 — novas features temporais

- Taxa de variação percentual de vibração e pressão (pct_change).
- Máximo e mínimo em janela móvel de 12 h para temperatura e vibração.
- Interação temp_x_vibracao = temperatura × vibração.

Tarefa 3 — SMOTE

O SMOTE foi aplicado **somente** em X_treino/y_treino (de 129 para 6.400 falhas sintéticas, base de treino balanceada em 50/50). A base de teste permaneceu intacta, evitando data leakage. No cenário C, os pesos de classe foram desligados, pois o SMOTE já compensa o desbalanceamento — usar os dois juntos corrigiria o problema em dobro. O StandardScaler da Regressão Logística foi ajustado apenas na base de treino.

3. Pergunta 1 — Impacto das novas variáveis no Recall dos Boostings

Modelo	Recall base (A)	Recall c/ novas features (B)	Variação	Falhas detectadas
XGBoost	2,00%	2,00%	+0 p.p.	1 → 1
CatBoost	8,00%	6,00%	-2 p.p.	4 → 3
LightGBM	20,00%	12,00%	-8 p.p.	10 → 6

Resposta: as novas variáveis **não melhoraram** o Recall dos modelos de Gradient Boosting — o efeito foi neutro no XGBoost e negativo no CatBoost e, principalmente, no LightGBM, que caiu de 20% para 12% (de 10 para 6 falhas detectadas).

Por quê. As features foram pensadas para capturar *tendências de degradação*, mas os dados sintéticos não têm degradação: cada hora é sorteada de forma independente e 96% das falhas vêm de um ruído aleatório de 2% que não depende dos sensores (no teste, só 3 das 50 falhas seguem a regra temperatura > 75 e vibração > 3,2). Sem sinal real, mais features dão aos modelos de árvore mais oportunidades de aprender padrões espúrios do treino que não se repetem no teste (sobreajuste). A feature temp_x_vibracao é a única ligada à regra física, mas a regra explica pouquíssimas falhas.

Nos modelos que não são de boosting, o efeito foi pequeno: Random Forest de 20,00% para 18,00% e Regressão Logística de 48,00% para 50,00%.

4. Efeito do SMOTE (cenário B → C)

Modelo	Recall B	Recall C	FP B → C	Custo B	Custo C
Random Forest	18,00%	30,00%	347 → 531	R$ 788.500,00	R$ 790.500,00
XGBoost	2,00%	12,00%	23 → 157	R$ 746.500,00	R$ 738.500,00
CatBoost	6,00%	14,00%	191 → 240	R$ 800.500,00	R$ 765.000,00
LightGBM	12,00%	16,00%	240 → 330	R$ 780.000,00	R$ 795.000,00
Regressão Logística	50,00%	50,00%	965 → 950	R$ 857.500,00	R$ 850.000,00

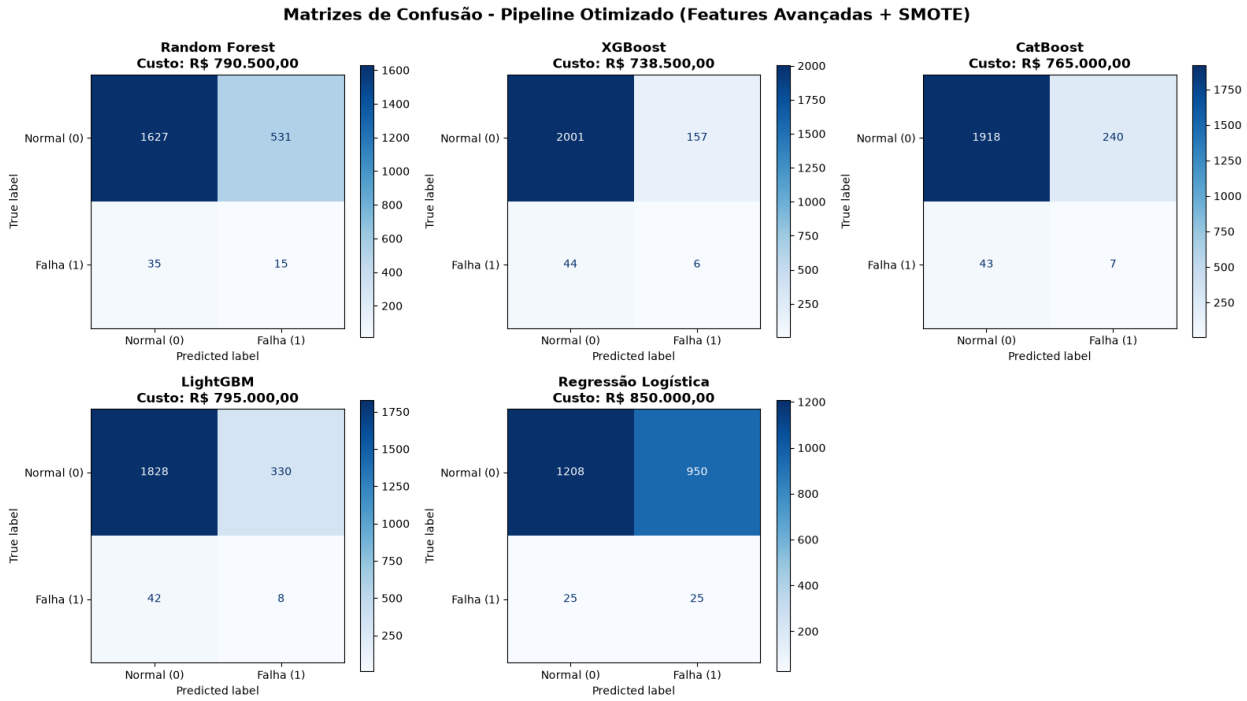
O SMOTE aumentou o Recall de quatro dos cinco modelos (XGBoost de 2% para 12%, CatBoost de 6% para 14%, Random Forest de 18% para 30%), porque o treino balanceado deixa os modelos mais dispostos a prever falha. Mas o ganho veio acompanhado de mais alarmes falsos, e o efeito no custo foi misto: melhorou XGBoost, CatBoost e Regressão Logística, e piorou Random Forest e LightGBM. Nesta base, SMOTE e pesos de classe têm desempenho equivalente — nenhuma das duas técnicas cria sinal onde ele não existe.

5. Tarefa 4 — Tabela de Custos Financeiros

Pipeline otimizado (cenário C: features avançadas + SMOTE)

Modelo	Recall	Precisão	Alarmes Falsos (FP)	Falhas Perdidas (FN)	Custo Financeiro Total
Random Forest	30,00%	2,75%	531	35	R$ 790.500,00
XGBoost	12,00%	3,68%	157	44	R$ 738.500,00
CatBoost	14,00%	2,83%	240	43	R$ 765.000,00
LightGBM	16,00%	2,37%	330	42	R$ 795.000,00
Regressão Logística	50,00%	2,56%	950	25	R$ 850.000,00

Linhas em verde: custo abaixo de 'nunca alarmar' (R$ 750.000,00).



Comparativo de custo de todos os cenários

Modelo	A - Base + Pesos	B - Otimizado + Pesos	C - Otimizado + SMOTE
Random Forest	R$ 849.500,00	R$ 788.500,00	R$ 790.500,00
XGBoost	R$ 752.500,00	R$ 746.500,00	R$ 738.500,00
CatBoost	R$ 823.500,00	R$ 800.500,00	R$ 765.000,00
LightGBM	**R$ 717.000,00**	R$ 780.000,00	R$ 795.000,00
Regressão Logística	R$ 894.000,00	R$ 857.500,00	R$ 850.000,00
Nunca alarmar (sem modelo)	R$ 750.000,00	R$ 750.000,00	R$ 750.000,00

De 15 combinações, apenas 3 custam menos do que não usar modelo algum. As demais geram tantos alarmes falsos que o custo da manutenção desnecessária supera a economia com as falhas evitadas: cada falha detectada economiza R$ 15.000, o que só compensa se vier acompanhada de menos de 30 alarmes falsos.

6. Recomendação: modelo a ser promovido para produção

Promover o LightGBM com as features originais e pesos de classe (cenário A). É a solução de menor custo total (R$ 717.000,00): detecta 10 das 50 falhas com 234 alarmes falsos, uma economia de R$ 33.000,00 (4,4%) em relação a operar sem modelo. O pipeline "otimizado" não deve substituí-lo.

Fundamentação. Em MLOps, uma mudança no pipeline (o *desafiante*) só deve ser promovida se superar o modelo em produção (o *campeão*) na métrica de negócio. Aqui, o melhor modelo do pipeline otimizado — XGBoost com SMOTE, R$ 738.500,00 — custa R$ 21.500,00 a mais que o LightGBM base. As novas features reduziram o Recall dos boostings e o SMOTE não compensou a perda. Modelos com Recall alto, como a Regressão Logística (50%), são os mais caros por causa do volume de alarmes falsos — Recall sozinho não é critério suficiente quando o custo de cada erro é conhecido.

Ressalvas. A vantagem é pequena e frágil: a diferença entre o melhor modelo e 'nunca alarmar' equivale a pouco mais de duas falhas (R$ 33.000,00), e a avaliação usa um único período com 50 falhas. Antes da promoção definitiva, recomenda-se:

- rodar o modelo em *shadow mode* (prevendo sem acionar manutenção) e monitorar o custo real;
- validar com TimeSeriesSplit em vários períodos para medir a estabilidade do resultado;
- ajustar o limiar de decisão pelo custo: com FN 30 vezes mais caro que FP, o limiar ótimo teórico é alarmar quando P(falha) > 500 / (500 + 15.000) ≈ 3,2%, desde que as probabilidades estejam calibradas;
- coletar dados reais com sinal de degradação — o principal limitador aqui é a base, não o algoritmo.

7. Conclusão

As Tarefas 2, 3 e 4 foram implementadas em app_manutencao_otimizado.py. As novas features temporais não melhoraram o Recall dos modelos de Gradient Boosting, o SMOTE trocou falhas perdidas por alarmes falsos sem ganho consistente de custo, e a análise financeira mostrou que o LightGBM do pipeline base é a opção mais barata para a TechIndustries. A principal lição é que otimizações de pipeline precisam ser validadas contra a métrica de negócio e contra uma referência sem modelo — complexidade adicional não garante resultado melhor.

Código-fonte, imagens e instruções de execução:
https://github.com/PedroZanette/atividade-manutencao-preditiva-mlops

Anexo — Tarefa 1: reprodução do ambiente base

Execução de app_manutencao.py (guia original, Passos 1 a 6) no ambiente virtual com Python 3.11, exibindo a tabela comparativa original no console:

```
pedro@pc: ~/projeto_manutencao_preditiva

(venv) pedro@pc:~/projeto_manutencao_preditiva$ python app_manutencao.py
--- [1/6] Gerando base de dados sintética de 1 ano (8.760 horas) ---
-> Base bruta salva em 'telemetria_maquina_2025.csv'

--- [2/6] Executando Engenharia de Features ---

--- [3/6] Divisão Temporal dos Dados ---
-> Base de teste (3 últimos meses) salva em 'dados_teste_ultimos_3_meses.csv'

--- [4/6] Treinando os 5 Modelos de Aprendizado de Máquina ---
-> Modelo: Random Forest treinado com sucesso!
-> Modelo: XGBoost treinado com sucesso!
-> Modelo: CatBoost treinado com sucesso!
-> Modelo: LightGBM treinado com sucesso!
-> Modelo: Regressão Logística treinado com sucesso!

--- [5/6] Tabela Comparativa de Métricas (Avaliação nos 3 Últimos Meses) ---

             Modelo Recall (Sensibilidade) Precisão F1-Score Acurácia  Alarmes Falsos (FP)  FALHAS PERDIDAS (FN)  Falhas Detectadas (TP)
      Random Forest                20.00%    1.96%    3.58%   75.59%                  499                    40                      10
            XGBoost                 2.00%    2.78%    2.33%   96.20%                   35                    49                       1
           CatBoost                 8.00%    1.48%    2.49%   85.82%                  267                    46                       4
           LightGBM                20.00%    4.10%    6.80%   87.59%                  234                    40                      10
Regressão Logística                48.00%    2.33%    4.44%   53.17%                 1008                    26                      24

--- [6/6] Exibindo e Salvando Painel Comparativo de Matrizes de Confusão ---
-> Imagem 'matriz_comparativa_5_modelos.png' salva com sucesso!
(venv) pedro@pc:~/projeto_manutencao_preditiva$
```

Painel matriz_comparativa_5_modelos.png gerado pelo pipeline base:

